Data Science 500 Assignment

RICHFIELD
richfield.ac.za

Name and Surname: Moshe Tholo

Student ITS No: 402422267

Qualification: BSc IT Year of Study: 1 Semester: 2nd

Assignment due date: 23 October 2026 Date submitted: 22 October 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents Introduction

Main Body (with relevant subheadings) Conclusion

References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature:  Date: 22 October 2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Jupyter Notebook Code

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression

print("All libraries imported successfully")
```

Base address of the site

BASE_URL = "https://books.toscrape.com/"

Download the first listing page

response = requests.get(BASE_URL) print("Status code:", response.status_code)

Parse the raw HTML into a searchable BeautifulSoup object

soup = BeautifulSoup(response.text, "html.parser")

Each book on the page sits inside an tag

books = soup.find_all("article", class_="product_pod") print("Books found on this page:", len(books))

Print the raw HTML of the first book so I can see its structure

print(books[0].prettify())

```
response.encoding = "utf-8"
soup = BeautifulSoup(response.text, "html.parser")
books = soup.find_all("article", class_="product_pod")
```

```
page_data = []

for book in books:
    title = book.h3.a["title"]
    price = book.find("p", class_="price_color").text
    rating = book.find("p", class_="star-rating")["class"][1]
    availability = book.find("p", class_="instock availability").text.strip()
    # Link to the book's own page;
    link = book.h3.a["href"]

    page_data.append({
        "title": title,
        "price": price,
        "rating": rating,
        "availability": availability,
        "link": link,
    })

df_page = pd.DataFrame(page_data)
df_page.head(10)

from urllib.parse import urljoin

# The links collected are relative (e.g. "catalogue/book_1000/index.html"),
# so urljoin combines them with the base address to make a full URL
first_link = urljoin(BASE_URL, df_page["link"][0])
print("Book page URL:", first_link)
```

```
book_response = requests.get(first_link)
book_response.encoding = "utf-8"
book_soup = BeautifulSoup(book_response.text, "html.parser")

breadcrumb_items = book_soup.find("ul", class_="breadcrumb").find_all("li")
genre = breadcrumb_items[2].text.strip()
print("Genre:", genre)

description_header = book_soup.find("div", id="product_description")
description = description_header.find_next_sibling("p").text if description_header else None
print("Description:", description[:100] if description else None)

# Product Information table: each row has a <th> label and a <td> value
info_table = {}
for row in book_soup.find("table", class_="table-striped").find_all("tr"):
    info_table[row.th.text] = row.td.text
print(info_table)

import time
from urllib.parse import urljoin

# One shared session: reuses the connection instead of repeating
# the slow SSL handshake on every request
session = requests.Session()
session.headers.update({"User-Agent": "Mozilla/5.0 (student data science project)"})
```

```
def get_soup(url, retries=3):
    """Download a page and return a BeautifulSoup object. Retries on failure."""
    for attempt in range(1, retries + 1):
        try:
            resp = session.get(url, timeout=15)
            resp.encoding = "utf-8"
            if resp.status_code == 200:
                return BeautifulSoup(resp.text, "html.parser")
            print(f"Attempt {attempt}/{retries}: status {resp.status_code} for {url}")
        except requests.RequestException:
            print(f"Attempt {attempt}/{retries}: request failed for {url}")
        time.sleep(2 * attempt)
    return None



def scrape_book_page(book_url):
    """Visit one book's own page and return its extra details."""
    soup = get_soup(book_url)
    if soup is None:
        return {}

    genre = soup.find("ul", class_="breadcrumb").find_all("li")[2].text.strip()

    desc_header = soup.find("div", id="product_description")
    description = desc_header.find_next_sibling("p").text if desc_header else None

    info = {}
```

```
    for row in soup.find("table", class_="table-striped").find_all("tr"):
        info[row.th.text] = row.td.text

    return {
        "genre": genre,
        "description": description,
        "upc": info.get("UPC"),
        "price_excl_tax": info.get("Price (excl. tax)"),
        "price_incl_tax": info.get("Price (incl. tax)"),
        "tax": info.get("Tax"),
        "availability_detail": info.get("Availability"),
        "num_reviews": info.get("Number of reviews"),
    }


def scrape_listing_page(page_number):
    """Scrape one listing page, then visit every book on it."""
    page_url = f"{BASE_URL}catalogue/page-{page_number}.html"
    soup = get_soup(page_url)
    if soup is None:
        return []

    records = []
    for book in soup.find_all("article", class_="product_pod"):
        record = {
            "title": book.h3.a["title"],
            "price": book.find("p", class_="price_color").text,
```

```
            "rating": book.find("p", class_="star-rating")["class"][1],
            "availability": book.find("p", class_="instock availability").text.strip(),
        }
        book_url = urljoin(page_url, book.h3.a["href"])
        record["url"] = book_url
        record.update(scrape_book_page(book_url))
        records.append(record)
        time.sleep(0.1)

    return records

all_books = []
start = time.time()

for page in range(1, 51):
    all_books.extend(scrape_listing_page(page))
    print(f"Page {page}/50 done, total books so far: {len(all_books)}")

    # Every 10 pages, save progress to disk.
    if page % 10 == 0:
        pd.DataFrame(all_books).to_csv("books_checkpoint.csv", index=False, encoding="utf-8-
sig")
        print(f"  Checkpoint saved at page {page}")

df_raw = pd.DataFrame(all_books)
print(f"Finished in {(time.time() - start) / 60:.1f} minutes. Shape: {df_raw.shape}")
```

```
missing = df_raw[df_raw["genre"].isna()]
print("Books missing detail data:", len(missing))

df_raw.to_csv("books_raw.csv", index=False, encoding="utf-8-sig")
print("Saved books_raw.csv")

test_data = []
for page in range(1, 3):
    test_data.extend(scrape_listing_page(page))
    print(f"Page {page} done, total books so far: {len(test_data)}")

df_test = pd.DataFrame(test_data)
df_test.head()

all_books = []

for page in range(1, 51):
    all_books.extend(scrape_listing_page(page))
    print(f"Page {page}/50 done, total books so far: {len(all_books)}")

df_raw = pd.DataFrame(all_books)
print("Final shape:", df_raw.shape)
```

index=False stops pandas adding an extra column of row numbers

utf-8-sig keeps the £ symbol readable if you open the file in Excel

df_raw.to_csv("books_raw.csv", index=False, encoding="utf-8-sig") print("Saved books_raw.csv")

df = pd.read_csv("books_raw.csv")

print("Shape:", df.shape)

print("\nData types:")

print(df.dtypes)

print("\nMissing values:")

print(df.isna().sum())

print("\nDuplicate UPCs:", df["upc"].duplicated().sum())

A look at the raw text formats that need cleaning

print("\nSample of raw values:")

print(df[["price", "rating", "availability", "availability_detail", "num_reviews"]].head())

#How many different values does each column hold?

for col in ["price", "price_excl_tax", "price_incl_tax", "tax", "num_reviews"]: print(col, "->", df[col].nunique(), "unique values")

print("\nPrice vs price_incl_tax identical?", (df["price"] == df["price_incl_tax"]).all())

df_clean = df.copy()

Remove duplicate books (UPC is a unique product code)

before = len(df_clean) df_clean = df_clean.drop_duplicates(subset="upc") print(f"Removed {before - len(df_clean)} duplicate rows")

Prices: strip everything except digits and the decimal point,

then convert to a float

for col in ["price", "price_excl_tax", "price_incl_tax", "tax"]: df_clean[col] = (df_clean[col].astype(str).str.replace(r"[^\d.]", "", regex=True).astype(float))

Rating: convert the word to a number so it can be used in a model

rating_map = {"One": 1, "Two": 2, "Three": 3, "Four": 4, "Five": 5} df_clean["rating"] = df_clean["rating"].map(rating_map)

stock: a simple 1/0 flag, plus the quantity pulled out of

text like "In stock (22 available)"

df_clean["in_stock"] = df_clean["availability"].str.contains("In stock").astype(int) df_clean["stock_quantity"] = (df_clean["availability_detail"].str.extract(r"((\d+) available)")[0].astype(float)) df_clean["stock_quantity"] = df_clean["stock_quantity"].fillna(0).astype(int)

Number of reviews: convert text to an integer

df_clean["num_reviews"] = pd.to_numeric(df_clean["num_reviews"], errors="coerce").fillna(0).astype(int)

Text fields: trim whitespace on title and genre

df_clean["title"] = df_clean["title"].str.strip() df_clean["genre"] = df_clean["genre"].str.strip()

Descriptions: remove the trailing "...more" some pages include,

flag which books have a description, and handle the missing ones

df_clean["description"] = df_clean["description"].str.replace(r"\s...more\s$", "", regex=True) df_clean["has_description"] = df_clean["description"].notna().astype(int) df_clean["description"] = df_clean["description"].fillna("No description available")

Text-derived features I can use later in the regression

df_clean["description_word_count"] = df_clean["description"].str.split().str.len() df_clean["title_length"] = df_clean["title"].str.len()

Drop columns that carry no extra information

```
df_clean = df_clean.drop(columns=["availability", "availability_detail", "price_excl_tax", "price_incl_tax", "tax"])

df_clean.info()


print("Shape:", df_clean.shape)

print("\nMissing values:\n", df_clean.isna().sum())

print("\nPrice summary:\n", df_clean["price"].describe())

print("\nRatings:\n", df_clean["rating"].value_counts().sort_index())

print("\nNumber of genres:", df_clean["genre"].nunique())

df_clean.head()


df_clean.to_csv("books_clean.csv", index=False, encoding="utf-8-sig")

print("Saved books_clean.csv")


plt.hist(df_clean["price"], bins=20, color="blue", edgecolor="black")

plt.title("Distribution of Book Prices")

plt.xlabel("Price (£)")

plt.ylabel("Number of books")

plt.show()


avg_price_by_rating = df_clean.groupby("rating")["price"].mean()


plt.bar(avg_price_by_rating.index, avg_price_by_rating.values, color="blue")

plt.title("Average Price by Star Rating")

plt.xlabel("Star rating")

plt.ylabel("Average price (£)")
```

plt.show()

top_genres = df_clean["genre"].value_counts().head(10)

plt.barh(top_genres.index, top_genres.values, color="blue")

plt.title("Top 10 Genres by Number of Books")

plt.xlabel("Number of books")

plt.show()

Scatter plot to see if stock quantity is related to price

plt.scatter(df_clean["stock_quantity"], df_clean["price"], color="blue", alpha=0.5) plt.title("Price vs Stock Quantity") plt.xlabel("Stock quantity") plt.ylabel("Price (£)") plt.show()

Correlation of each numeric column with price

numeric_data = df_clean[["price", "rating", "stock_quantity", "num_reviews", "description_word_count", "title_length"]]

print(numeric_data.corr()["price"])

Tools for splitting the data and measuring the model

from sklearn.model_selection import train_test_split from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score import numpy as np

Numeric features to use (num_reviews is left out because it is always 0)

numeric_features = ["rating", "stock_quantity", "description_word_count", "title_length", "has_description"]

Genre is text, convert it into 0/1 columns (one column per genre)

genre_columns = pd.get_dummies(df_clean["genre"], prefix="genre", drop_first=True)

X = the inputs, y = the thing we want to predict

X = pd.concat([df_clean[numeric_features], genre_columns], axis=1).astype(float) y = df_clean["price"]

print("Number of features:", X.shape[1]) print("Number of books:", X.shape[0])

Keep 20% of the books hidden so I can test the model on data it has not seen

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Train the linear regression model

model = LinearRegression() model.fit(X_train, y_train)

print("Model trained on", len(X_train), "books, tested on", len(X_test), "books")

Predict prices for the test books

y_pred = model.predict(X_test)

Measure how good the predictions are

r2 = r2_score(y_test, y_pred) mae = mean_absolute_error(y_test, y_pred) rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"R-squared: {r2:.3f}") print(f"Mean Absolute Error: £{mae:.2f}") print(f"Root Mean Squared Error: £{rmse:.2f}")

Compare with a very simple baseline: always guess the average price

baseline_pred = [y_train.mean()] * len(y_test) print(f"Baseline MAE (always guessing the average): £{mean_absolute_error(y_test, baseline_pred):.2f}")

Show the 10 features with the biggest effect on predicted price

coefficients = pd.Series(model.coef_, index=X.columns) print(coefficients.sort_values(key=abs, ascending=False).head(10))

Compare three versions of the model on the same train/test split

def evaluate(features, label): X_all = features.astype(float) X_tr, X_te, y_tr, y_te = train_test_split(X_all, y, test_size=0.2, random_state=42) m = LinearRegression().fit(X_tr, y_tr)

pred = m.predict(X_te) print(f"{label:<35} R2 = {r2_score(y_te, pred):7.3f} MAE = £{mean_absolute_error(y_te, pred):.2f}")

evaluate(df_clean[numeric_features], "Numeric features only") evaluate(genre_columns, "Genre only") evaluate(pd.concat([df_clean[numeric_features], genre_columns], axis=1), "Numeric + genre (original model)")

print("Number of cleaned books:", len(df_clean))

print("\nPrice statistics:")

print(df_clean["price"].describe())

print("\nAverage price by rating:")

print(df_clean.groupby("rating")["price"].mean())

print("\nTop 10 genres:")

print(df_clean["genre"].value_counts().head(10))

print("\nCorrelation with price:")

print(numeric_data.corr()["price"])

Look at the books with these "genres" and their page URLs

suspect = df_clean[df_clean["genre"].isin(["Default", "Add a comment"])]
print(suspect["genre"].value_counts()) suspect[["title", "genre", "url"]].head()

Book Market Analytics

THOLO MOSHE

402422267

Data Science 500

23 October 2026

Executive Summary

This project looked at book prices and other data from the Books to Scrape website. Python was used to gather data, organize the dataset, make charts and graphs, and develop a linear regression model to forecast book prices.

One thousand book records were gathered from the website. On average, books cost £35.07 each, and the prices varied from £10.00 up to £59.99. The analysis looked at prices along with star ratings, how many items were in stock, the book categories, and other details.

The results showed that the numerical features tested had very weak connections with price. The linear regression model did not do as well as a basic method that just guessed the average price. This means the information I had wasn't enough to guess the prices correctly.

This project shows how web scraping and data analysis can be used to look into a book catalog. It also shows how important it is to make sure the data is accurate, check how well the model works, and choose the right features before making any predictions.

Introduction

Book prices can change based on different things like the kind of book, who published it, the format it's in, and other features. Looking at information from a book catalog can help stores and publishers see how prices are set.

The purpose of this project was to look at book data from Books to Scrape, a site that's meant for practicing web scraping. The project used Python to gather and organize the data, look for patterns with graphs, and create a model to predict book prices.

The main goals were to gather book details from the website, organize the data, look into how prices connect with other factors, and check the performance of a linear regression model.

The project aimed to identify the weaknesses in the data suggest ways to enhance future price prediction.

Methodology

Data Collection

By utilizing the Requests and BeautifulSoup libraries in Python, I was able to scrape data from all 50 catalogue pages on Books to Scrape. I gathered 1,000 books, and for each one, I noted down the title, price, rating, availability, genre, and description. The data was stored in a CSV file.

Data Cleaning

I used Pandas to delete duplicate entries, turn prices and ratings into numbers, and get ready for stock data. Additionally, I cleaned text fields and enhanced features like title length and description word count. The cleaned dataset was saved as books_clean.csv.

Analysis and Modelling

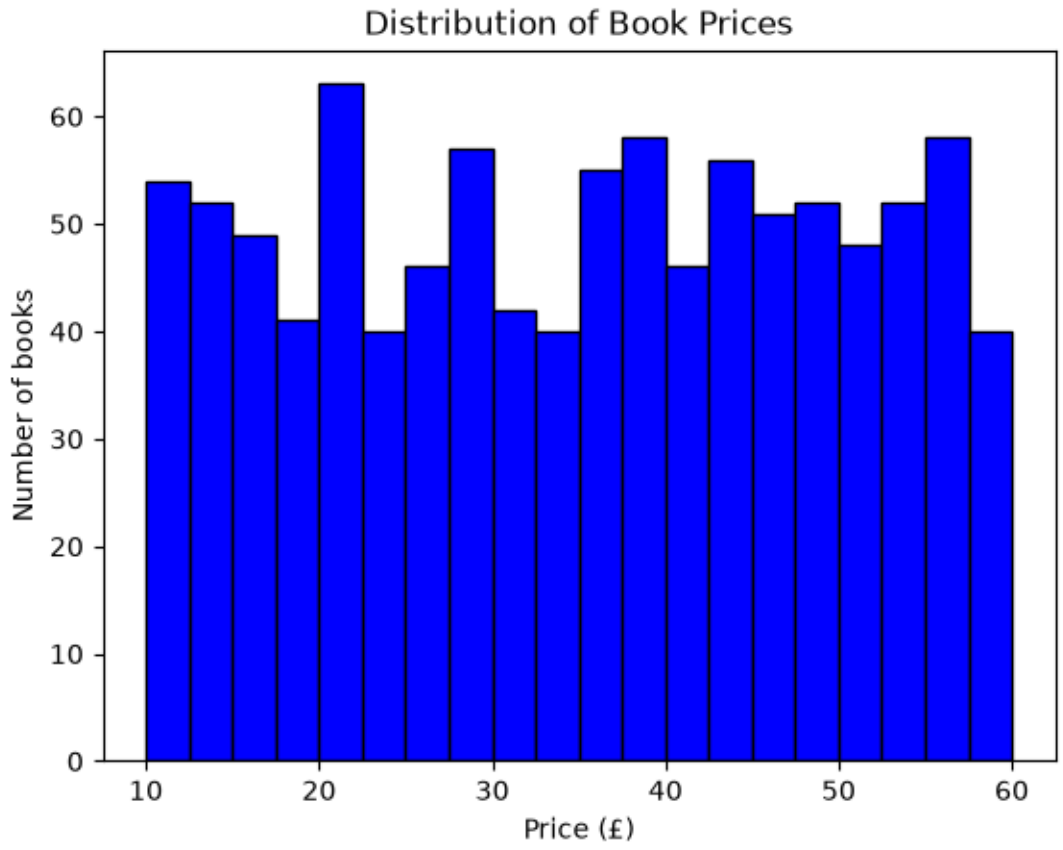
I made graphs using Matplotlib and Seaborn to show how prices are spread out, the average price for each rating, how many books are in each genre, and how price relates to how many copies are in stock.

Using an 80/20 training and testing split, I proceeded to train a linear regression model. Its performance was compared to a baseline model that predicted the average price.

Results

Book Prices

The median was £35.98, and the average price was £35.07. Prices ranged from £10.00 to £59.99. Half of the books fall in a price range from £22.11 to £47.46.



Ratings and Prices

The average cost of four-star books was £36.09, which was higher than the average price of £34.56 for one-star books. The small difference shows that ratings weren't closely connected to price.

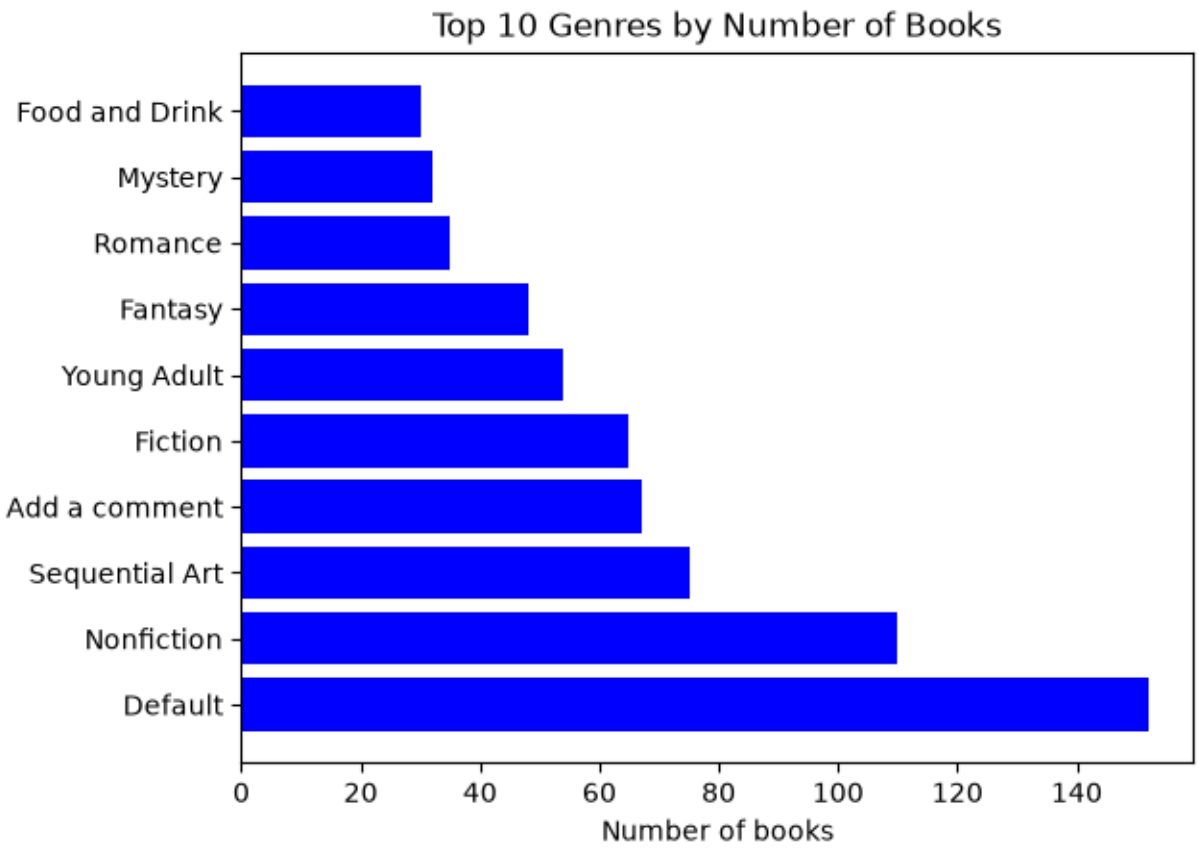
The figure below displays the average price based on a star rating.



Genres and Stock

The genre graph showed that some labels, like "Default" and "Add a comment," weren't real genres. category data was scraped incorrectly, this suggests.

The link between how much stock there is and its price was about -0.011, which means there's almost no straight-line connection between them.


Top 10 Genres by Number of Books
Food and Drink
Mystery
Romance
Fantasy
Young Adult
Fiction
Add a comment
Sequential Art
Nonfiction
Default
0
20
40
60
80
100
120
140
Number of books



Predictive Model

The links between price and rating, stock quantity, description word count, and title length were all almost zero.

The MAE in the linear regression model was £13.08, whereas the baseline MAE was £12.66. The model therefore didn't do better than just guessing the average price.

The three model versions all had negative R-squared values: -0.018 for the numerical features, -0.100 for the genre-only model, and -0.111 for the combined features.

Limitations and Recommendations

The project's results may not be representative of the broader book market, as it was based on a practice website. The wrong genre categories and a column for reviews that only had zeros made the analysis harder.

Future work should focus on improving the quality of the data and gather more helpful information, like the publisher, when the book was published, how many pages it has, and the type of format it's in. Other models can then be compared to the baseline.

Retailers should not use this model to decide prices because the predictions it made were not accurate enough.

Conclusion

This project looked into book prices for 1,000 books by using web scraping and data analysis. The results showed that average prices were about the same in each group of ratings and that the chosen features didn't strongly connect with price.

The linear regression model did not perform better than the baseline. The development of a more useful price prediction model would necessitate more informative features and better-quality data.

Reference List

Books to Scrape (n.d.) *Books to Scrape*. Available at: https://books.toscrape.com/ (Accessed: [09 October 2026]).

Beautiful Soup (n.d.) *Beautiful Soup Documentation*. Available at: https://www.crummy.com/software/BeautifulSoup/bs4/doc/ (Accessed: [11 October 2026]).

Pandas (n.d.) *Pandas Documentation*. Available at: https://pandas.pydata.org/docs/ (Accessed: [11 October 2026]).

Scikit-learn (n.d.) *Scikit-learn Documentation*. Available at: https://scikit-learn.org/stable/ (Accessed: [19 October 2026).

Matplotlib (n.d.) *Matplotlib Documentation*. Available at: https://matplotlib.org/stable/ (Accessed: [11 October 2026]).

Scikit-learn (n.d.) *Scikit-learn Documentation*. Available at: https://scikit-learn.org/stable/ (Accessed: [18 October 2026]).